STEP 1: CREATE MAVEN PROJECT

```
project/
│
├── src/
│   ├── main/java/
│   └── test/java/
│
├── pom.xml
```

STEP 2: WRITE JAVA CODE (INHERITANCE)

App.java

```
// Base class
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    void sound() {
        System.out.println("Animal makes sound");
    }
}

// Derived class (Inheritance)
class Dog extends Animal {

    Dog(String name) {
        super(name);
    }

    // Method overriding
    void sound() {
        System.out.println(name + " barks");
    }
}

// Main class
public class App {
    public static void main(String[] args) {
        Dog d = new Dog("Tommy");  // You can change name here
        d.sound();
    }
}
```

STEP 3: WRITE JUNIT TESTS

AppTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class AppTest {

    @Test
    void testObjectCreation() {
        Dog d = new Dog("Tommy");
        assertEquals("Tommy", d.name);
    }

    @Test
    void testInheritance() {
        Dog d = new Dog("Tommy");
        assertTrue(d instanceof Animal);
    }
}
```

STEP 4: CONFIGURE

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.example</groupId> <!-- CHANGE -->
 <artifactId>my-app</artifactId> <!-- CHANGE -->
 <version>1.0</version>

 <properties>
   <maven.compiler.source>17</maven.compiler.source>
   <maven.compiler.target>17</maven.compiler.target>
 </properties>

 <dependencies>
   <dependency>
     <groupId>org.junit.jupiter</groupId>
     <artifactId>junit-jupiter</artifactId>
     <version>5.10.0</version>
     <scope>test</scope>
   </dependency>
 </dependencies>

 <build>
```

```
   <plugins>
     <plugin>
       <groupId>org.apache.maven.plugins</groupId>
       <artifactId>maven-surefire-plugin</artifactId>
       <version>3.2.5</version>
     </plugin>
   </plugins>
 </build>

</project>
```

###Replace:

com.example → your name/company (e.g., com.lakshaya)
my-app → your project name###

STEP 5: GITHUB COMMANDS

```
git init
git add .
git commit -m "Initial commit"
git branch -M main

# REPLACE BELOW URL
git remote add origin https://github.com/USERNAME/REPO-NAME.git

git push -u origin main
```

###Replace:

USERNAME → your GitHub username
REPO-NAME → your repository name###

STEP 6: MODIFY CODE & PUSH AGAIN

###Make any change (example):

```
System.out.println("Dog is barking loudly");
```

###Then:

```
git add .
git commit -m "Updated code"
git push
```

STEP 7: DOCKERFILE

dockerfile:

```
FROM eclipse-temurin:17
WORKDIR /app
COPY target/*.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

STEP 8: BUILD & TEST

```
mvn clean test
mvn package
```

STEP 9: DOCKER COMMANDS

```
# REPLACE IMAGE NAME
docker build -t USERNAME/app-name:latest .

docker push USERNAME/app-name:latest
```

Replace:

USERNAME → DockerHub username
app-name → project name

STEP 10: JENKINS PIPELINE

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "USERNAME/app-name:latest"  // CHANGE
    }

    stages {

        stage('Checkout GitHub') {
            steps {
                git 'https://github.com/USERNAME/REPO-NAME.git' // CHANGE
            }
        }

        stage('Test using JUnit') {
            steps {
                sh 'mvn clean test'
            }
        }
```

```
        stage('Build using Maven') {
            steps {
                sh 'mvn package'
            }
        }

        stage('Create Docker Image') {
            steps {
                sh 'docker build -t $DOCKER_IMAGE .'
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $DOCKER_IMAGE'
            }
        }
    }
}
```

###Replace:

USERNAME → your GitHub & DockerHub username
REPO-NAME → GitHub repo name
app-name → project name

PART (b): INHERITANCE EXPLANATION

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

◇ Explanation
Animal → Parent class
Dog → Child class
extends keyword used
Method overriding implemented

Code reuse achieved